



*ugr*

Universidad  
de Granada

## SEMINARIO 3

# INTRODUCCIÓN AL USO DE ARDUINO Y SIMULADOR TINKERCAD

Autor:

Mario Casas Pérez

## ÍNDICE

1. [Ejercicio 1 físico](#)
2. [Ejercicio 2 físico](#)
3. [Ejercicio 1 con simulador Tinkercad](#)
4. [Ejercicio 2 con simulador Tinkercad](#)
5. [Bibliografía](#)

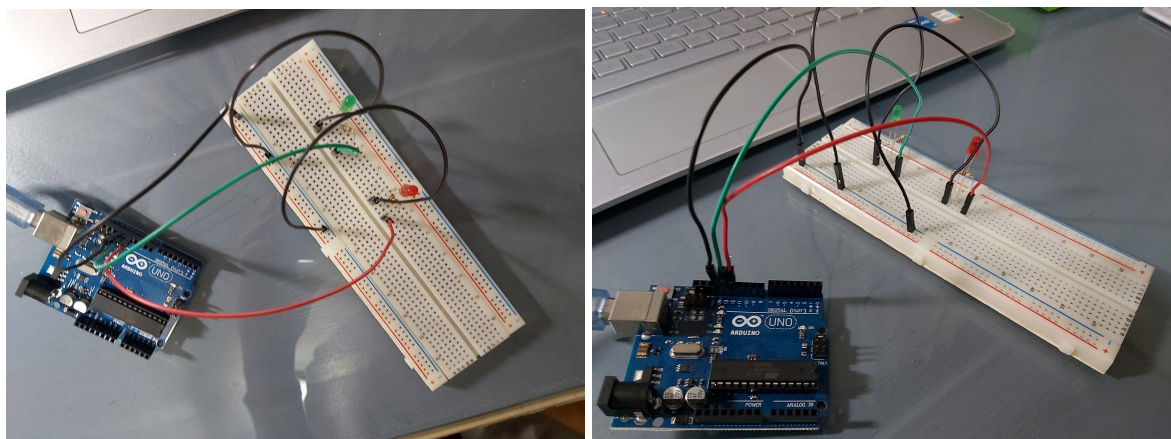
## 1. Ejercicio 1 físico

El primer ejercicio trata de implementar el programa de parpadeo LED ampliándolo para que encienda y apague alternativamente dos LED (uno rojo y otro verde), conectados a las salidas digitales 12 y 13 del Arduino, a un intervalo de 1.5 segundos. Se deberá, además, crear el esquema con Fritzing y cargar el programa en Arduino para comprobar que funciona correctamente.

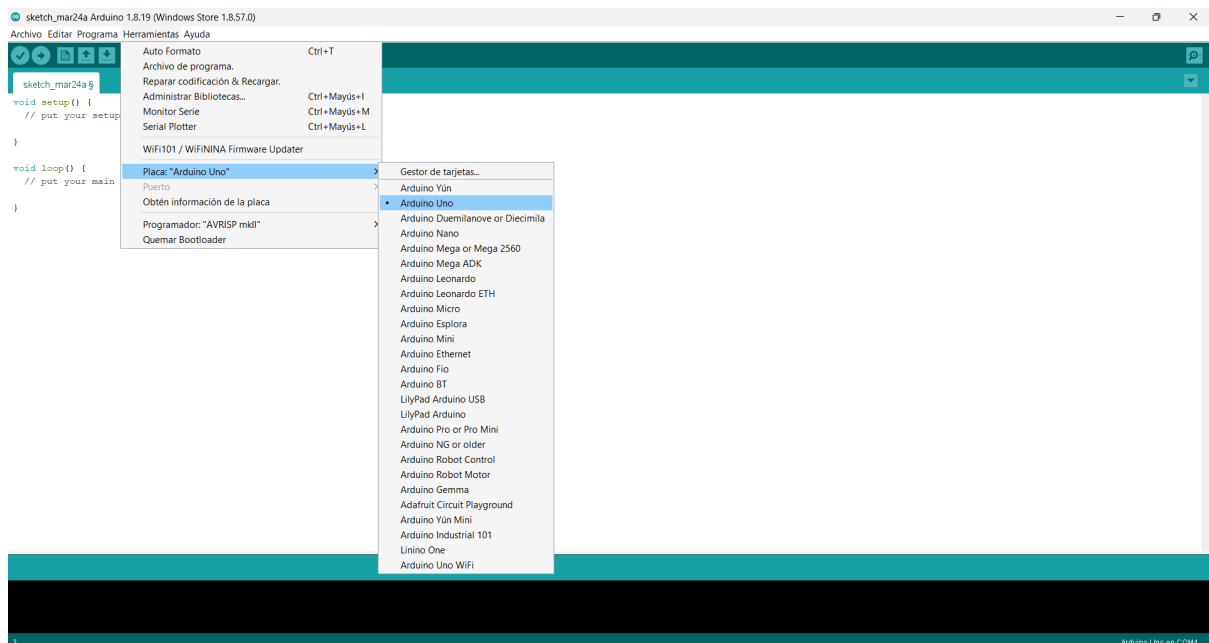
Los componentes usados son:

- 1 LED verde.
- 1 LED rojo.
- 2 resistencias de 1 k $\Omega$ .
- Arduino UNO R3.
- Placa de pruebas.
- 5 cables conectores.

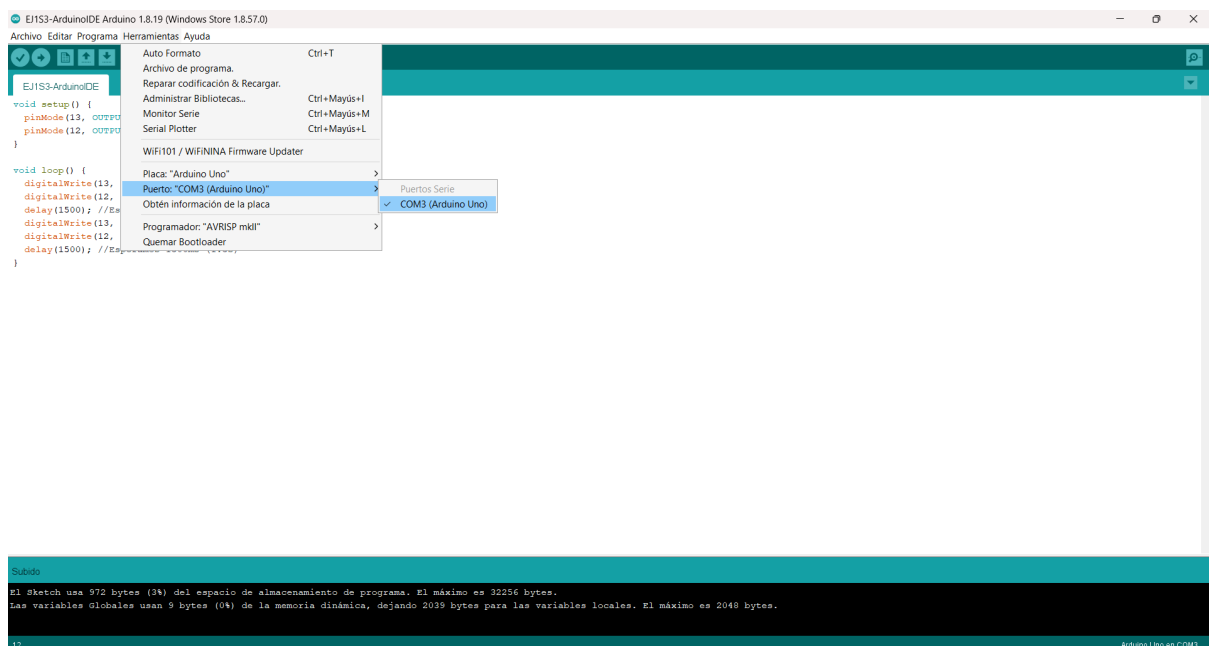
Lo primero de todo es realizar el ejercicio con una placa de pruebas y con un *Arduino UNO R3*, montándolo con los componentes correspondientes. El ejercicio montado quedaría de la siguiente manera:



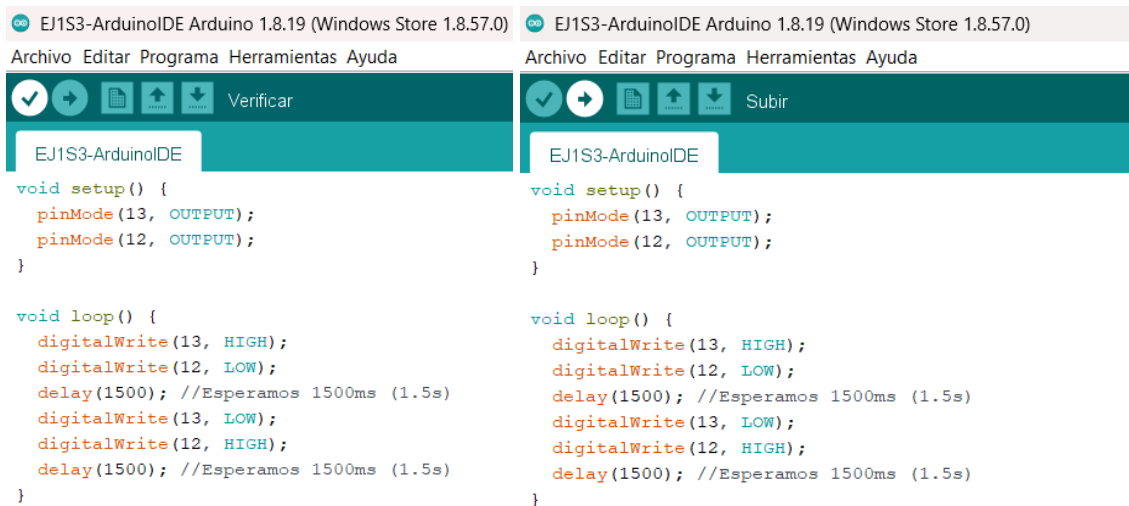
Vamos a usar el programa Arduino IDE para poder implementar las funciones que queremos que se hagan cuando ya tengamos construido el ejercicio 1. Para ello, lo primero de todo es configurar dicho programa para que se conecte a la placa. Nos vamos a la parte del menú donde pone Herramientas y en *Placa* seleccionamos *Arduino Uno*.



Además, cuando conectemos la placa, debemos de especificar el puerto que se va a utilizar (COM3 Arduino 1).



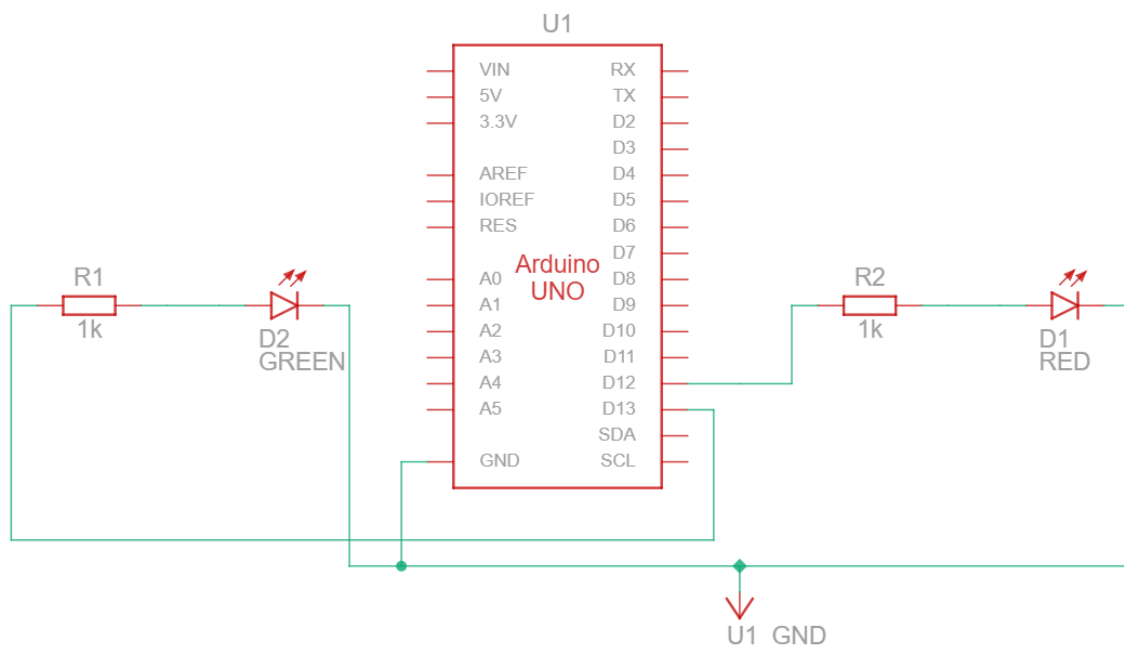
Luego, debemos de cargar el programa. Para ello primero debemos de pinchar el el botón de *Verificar* que está en la parte superior izquierda y luego a *Subir* (al lado de *Verificar*) para que el programa empiece a funcionar en la placa.



En el primer método (setup) lo que hacemos es configurar los pines digitales 12 y 13 como salida, lo que nos permite poder enviar señales para encender y apagar los LED.

En el segundo método (loop) se controla estos pines digitales 12 y 13 de forma alternada para que se produzca el parpadeo, por lo que primeramente encendemos el pin 13 mientras que el pin 12 está apagado y, tras 1500 milisegundos, encendemos el pin 12 y apagamos el pin 13 para luego en otros 1500 milisegundos encender el pin contrario y apagar el que estaba encendidos. Esto se hace indefinidamente.

El esquema con Fritzing es el siguiente:



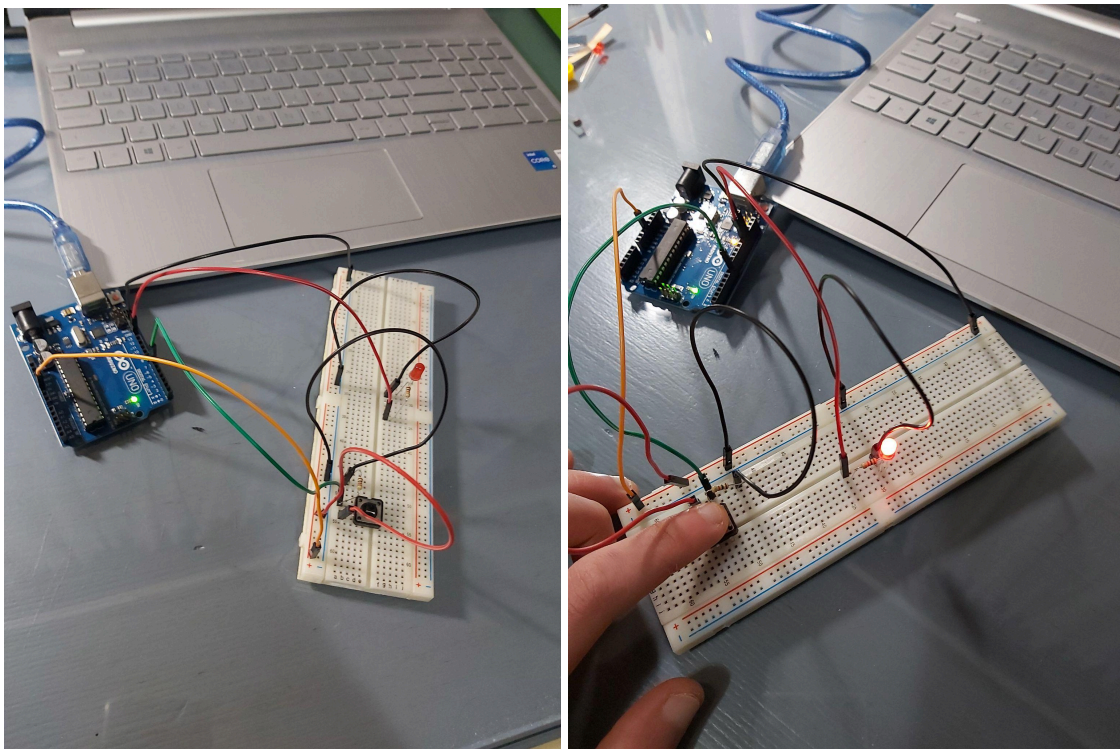
## 2. Ejercicio 2 físico

El segundo ejercicio trata de implementar el programa de parpadeo de LED ampliándolo con las modificaciones necesarias para que se encienda el LED solo cuando se pulse un interruptor conectado a la entrada digital 7.

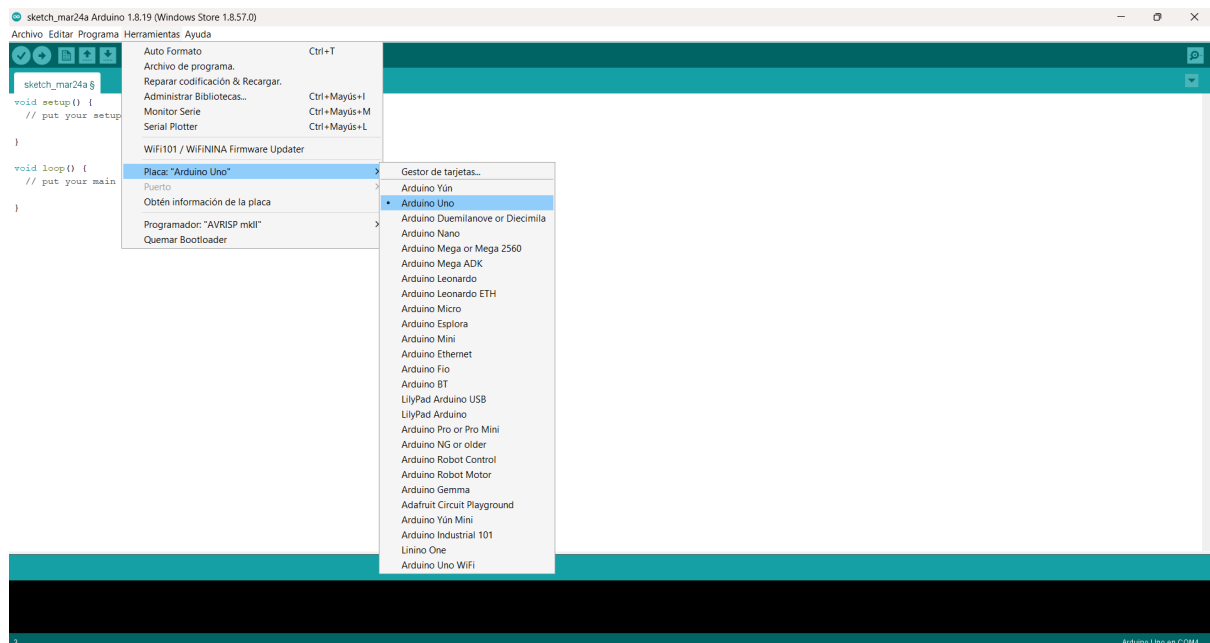
Los componentes usados son:

- 1 LED rojo.
- 1 pulsador
- 2 resistencias de 1 kΩ.
- Arduino UNO R3.
- Placa de pruebas.
- 7 cables conectores.

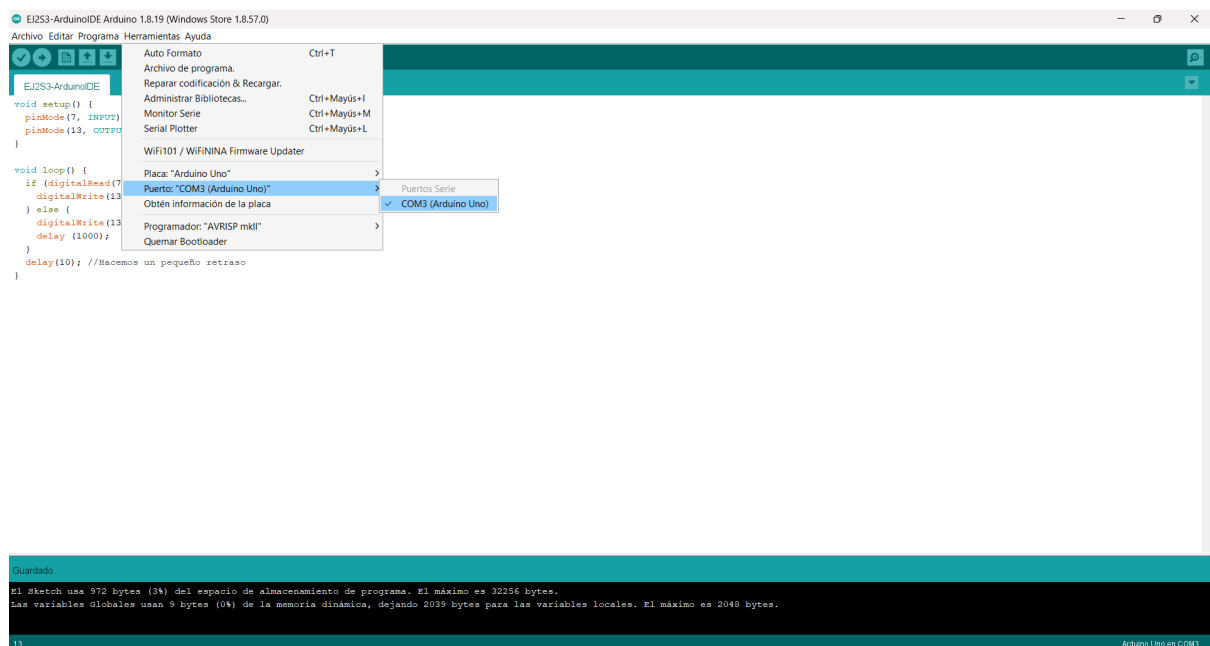
Lo primero de todo es realizar el ejercicio con una placa de pruebas y con un *Arduino UNO R3*, montándolo con los componentes correspondientes. El ejercicio montado quedaría de la siguiente manera:



Para este ejercicio también se ha usado el programa Arduino IDE para poder implementar las funciones que queremos que se hagan cuando ya lo tengamos construido. Para ello, lo primero de todo es configurar dicho programa de la misma forma que en el ejercicio anterior para que se conecte a la placa. Nos vamos a la parte del menú donde pone Herramientas y en *Placa* seleccionamos *Arduino Uno*.

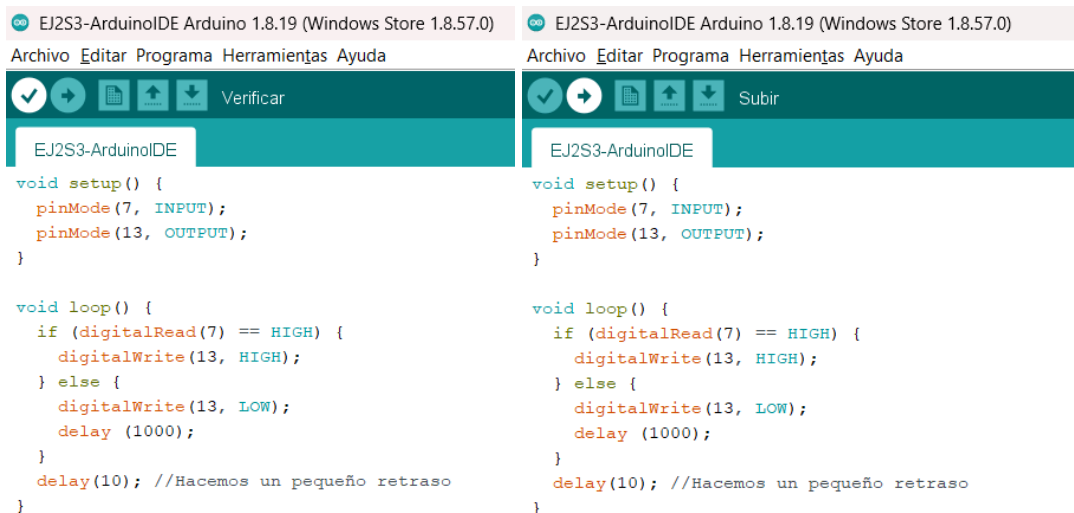


Además, cuando conectemos la placa, debemos de especificar el puerto que se va a utilizar (COM3 Arduino 1).



Luego, debemos de cargar el programa. Para ello primero debemos de pinchar el el botón de *Verificar* que está en la parte superior izquierda y luego a *Subir* (al lado de *Verificar*) para que el programa empiece a funcionar en la placa.

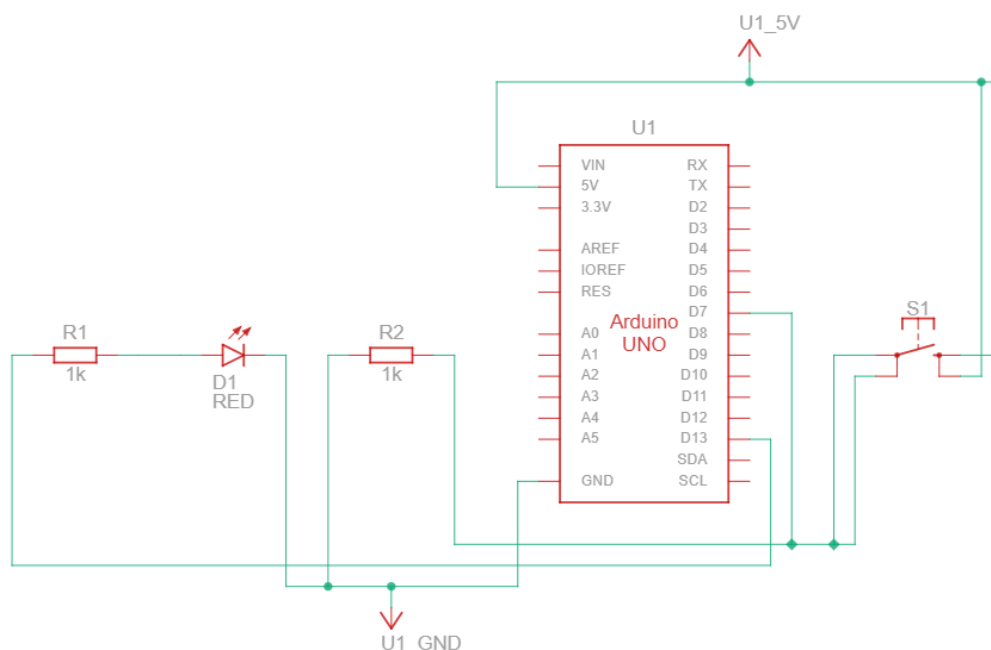




En el primer método (setup) lo que hacemos es configurar el pin digital 13 como salida, lo que nos permite poder enviar señales para encender y apagar el LED, mientras que el pin digital 7 lo configuramos como entrada para que pueda leer señales de este pin, como lo es los datos que le transmita nuestro pulsador.

En el segundo método (loop) establecemos una relación entre el pin de entrada 7 y el pin de salida 13. Lo que hacemos es leer el estado del pin digital 7 y, si detecta una señal HIGH, entonces se enciende el pin 13. Si el pin 7 no está en HIGH, entonces el pin 13 se apaga. Además, se hace una espera de 10 milisegundos para mejorar el rendimiento del simulador. Esto se hace indefinidamente.

El esquema con Fritzing es el siguiente:



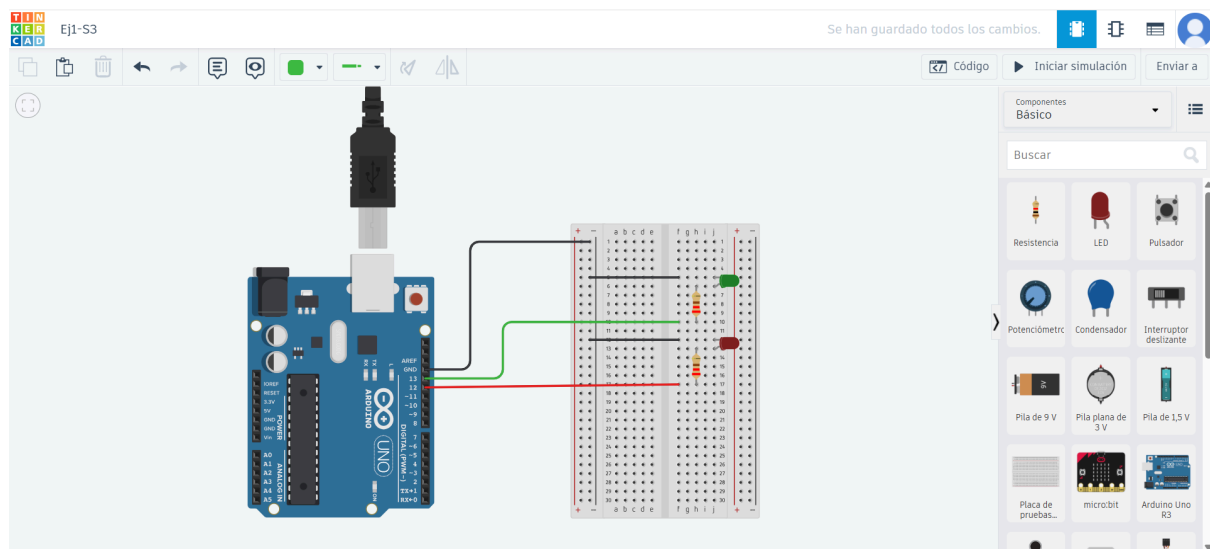


### 3. Ejercicio 1 con simulador Tinkercad

Recordemos que el ejercicio 1 trataba de, a partir del programa de parpadeo de LED, ampliarlo para que encienda y apague de forma alternativa dos LED (uno rojo y otro verde), conectados a las salidas digitales 12 y 13 del Arduino, teniendo un intervalo de 1.5 segundos (1500 milisegundos).

La realización de este ejercicio se va a llevar a cabo mediante el uso del simulador *Tinkercad*.

Se ha cogido un *Arduino UNO R3* y una placa de pruebas para llevar a cabo esta tarea. Los LED están conectados a esta placa en donde los cátodos están conectados a tierra y los ánodos a las salidas digitales 12 y 13. Se ha empleado una resistencia de 220Ω en cada ánodo de los LED para evitar sobretensiones en los diodos LED. El esquema es el siguiente:



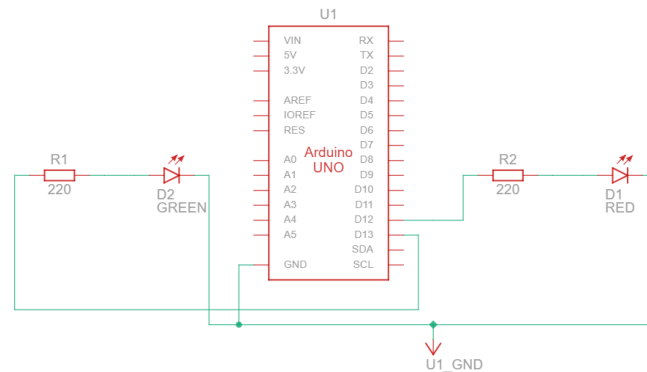
El código empleado para que se alterne el encendido de los LED es el siguiente:

```
1 void setup()
2 {
3   pinMode(13, OUTPUT);
4   pinMode(12, OUTPUT);
5 }
6
7 void loop()
8 {
9   digitalWrite(13, HIGH);
10  digitalWrite(12, LOW);
11  delay(1500); //Esperamos 1500ms (1.5s)
12  digitalWrite(13, LOW);
13  digitalWrite(12, HIGH);
14  delay(1500); //Esperamos 1500ms (1.5s)
15 }
```

En el primer método (setup) lo que hacemos es configurar los pines digitales 12 y 13 como salida, lo que nos permite poder enviar señales para encender y apagar los LED.

En el segundo método (loop) se controla estos pines digitales 12 y 13 de forma alternada para que se produzca el parpadeo, por lo que primeramente encendemos el pin 13 mientras que el pin 12 está apagado y, tras 1500 milisegundos, encendemos el pin 12 y apagamos el pin 13 para luego en otros 1500 milisegundos encender el pin contrario y apagar el que estaba encendidos. Esto se hace indefinidamente.

El esquema con Fritzing es el siguiente:



Para ver el proyecto desde Tinkercad, usar el enlace

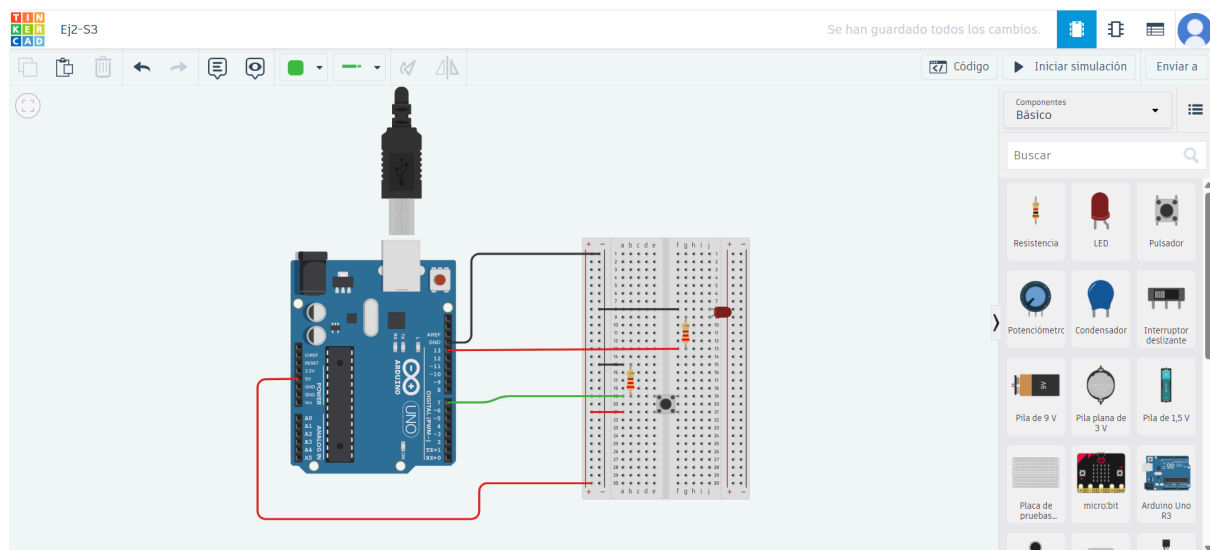
[https://www.tinkercad.com/things/dbgGSlymrHW-ej1-s3?sharecode=72KMsqeXTfGCfvo2wH\\_g66ymFf\\_FU1SziaYWR6uSQDY](https://www.tinkercad.com/things/dbgGSlymrHW-ej1-s3?sharecode=72KMsqeXTfGCfvo2wH_g66ymFf_FU1SziaYWR6uSQDY)

## 4. Ejercicio 2 con simulador Tinkercad

El ejercicio 2 trataba de implementar el programa de parpadeo de LED ampliándolo de manera que se encienda el LED solo cuando se pulse un interruptor conectado a la entrada digital 7.

La realización de este ejercicio se va a llevar a cabo mediante el uso del simulador *Tinkercad*.

Se ha cogido un *Arduino UNO R3* y una placa de pruebas para llevar a cabo esta tarea. El LED está conectado a esta placa en donde el cátodo está conectado a tierra y el ánodo a la salida digital 13. Se ha empleado una resistencia de  $220\Omega$  en el ánodo del LED para evitar sobretensiones en el diodo LED. Además, se ha conectado un pulsador a la placa de pruebas en donde una de las patillas la tiene conectada a tierra y al pin digital 7, mientras que a la otra se le ha conectado un voltaje de 5V. El esquema es el siguiente:



El código empleado para que se alterne el encendidos de los LED es el siguiente:

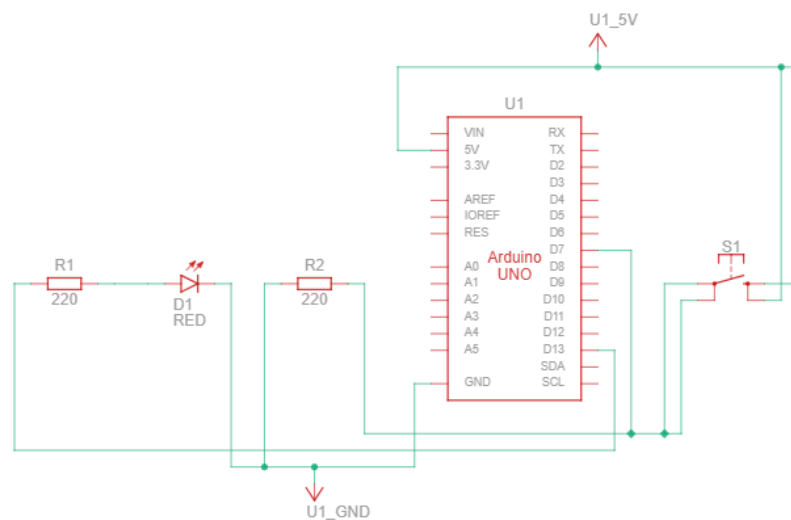
```
1 void setup()
2 {
3   pinMode(7, INPUT);
4   pinMode(13, OUTPUT);
5 }
6
7 void loop()
8 {
9   if (digitalRead(7) == HIGH) {
10    digitalWrite(13, HIGH);
11   } else {
12    digitalWrite(13, LOW);
13    delay (1000);
14   }
15   delay(10); //Hacemos un pequeño retraso
16 }
```

En el primer método (setup) lo que hacemos es configurar el pin digital 13 como salida, lo que nos permite poder enviar señales para encender y apagar el LED, mientras que el pin

digital 7 lo configuramos como entrada para que pueda leer señales de este pin, como lo es los datos que le transmita nuestro pulsador.

En el segundo método (loop) establecemos una relación entre el pin de entrada 7 y el pin de salida 13. Lo que hacemos es leer el estado del pin digital 7 y, si detecta una señal HIGH, entonces se enciende el pin 13. Si el pin 7 no está en HIGH, entonces el pin 13 se apaga. Además, se hace una espera de 10 milisegundos para mejorar el rendimiento del simulador. Esto se hace indefinidamente.

El esquema con Fritzing es el siguiente:



Para ver el proyecto desde Tinkercad, usar el enlace

[https://www.tinkercad.com/things/l8adsycWJcn-ej2-s3?sharecode=MBYf6F68G84WfPqUMA ug6WKj11-bpnH\\_lqHsL-ZYgfA](https://www.tinkercad.com/things/l8adsycWJcn-ej2-s3?sharecode=MBYf6F68G84WfPqUMA ug6WKj11-bpnH_lqHsL-ZYgfA)

## 5. Bibliografía

<https://swad.ugr.es/swad/tmp/8O/PHmXfsR9IN94Ayw0n53wztagBMdyZy0lCHdsrlgCQ/S3-Arduino.pdf>

<https://swad.ugr.es/swad/tmp/Q7/hseymnValX0-x0aXsYmx8wN912o6FQ5ibt9NijX1l/S3-simulador-Arduino.pdf>